

LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III



TUGAS PRAKTIKUM ACARA 10

Nama:
RAJATA PRINGGIT BINTANG BIHAR

NIM:
E41250264

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada aplikasi Acara 9, beberapa Controller masih menulis method yang sama berulang kali, seperti `redirect()` dan `flash()`, serta memanggil tampilan dengan path file yang lengkap. Model statis juga membuat koneksi PDO sendiri, terpisah dari class Database. Duplikasi seperti ini membuat perubahan kecil harus dilakukan di banyak tempat.

Praktikum Acara 10 membahas Inheritance dan Repository Pattern untuk mengatasinya. Inheritance memungkinkan method yang sama ditulis sekali di `BaseController` dan diwarisi oleh semua Controller, sedangkan Repository Pattern memusatkan seluruh query SQL di Repository sehingga Controller hanya memanggil method yang tersedia.

2. Tujuan

1. Membuat `BaseController` yang berisi method `view()` dan `redirect()`.
2. Membuat `MahasiswaController` yang mewarisi `BaseController` dan memakai method yang diwariskan.
3. Membuat `MahasiswaRepository` yang menangani seluruh operasi database mahasiswa.
4. Memastikan `MahasiswaController` mengakses data hanya melalui repository, tanpa query SQL.
5. Menguji fitur tampil, tambah, ubah, dan hapus data mahasiswa.

3. Dasar Teori

Inheritance

Inheritance (pewarisan) adalah konsep OOP yang memungkinkan sebuah class mewarisi atribut dan method dari class lain. Dengan inheritance, kode yang sama tidak perlu ditulis berulang kali sehingga lebih mudah dipelihara. Keuntungannya adalah berkurangnya duplikasi kode (prinsip DRY), pemeliharaan lebih mudah, dan perubahan cukup dilakukan pada class induk.

Base Controller dan Base Model

Pada aplikasi MVC, banyak Controller memiliki fungsi yang sama, misalnya menampilkan halaman dan mengarahkan pengguna ke halaman lain. Fungsi tersebut ditempatkan pada

BaseController yang diwarisi semua Controller. Dengan cara yang sama, BaseModel menyediakan akses PDO bagi Model yang membutuhkannya.

Repository Pattern

Repository Pattern adalah pola desain yang mengelola seluruh proses akses data ke database. Seluruh perintah SQL ditempatkan di Repository sehingga Controller tidak mengetahui bagaimana query dijalankan. Jika struktur database atau DBMS berubah, perubahan cukup dilakukan pada Repository.

Hasil Praktik

2. Implementasi

a. BaseController

BaseController (app/Core/BaseController.php) berisi method view() untuk menampilkan halaman, redirect() untuk mengarahkan pengguna, dan flash() untuk menyimpan pesan sementara. Ketiganya ditulis satu kali dan diwarisi oleh semua Controller.

```
class BaseController
{
    protected function view(
        string $view,
        array $data = [],
        string $active = '',
        bool $useLayout = true
    ): void {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        extract($data, EXTR_SKIP);

        $content = __DIR__ . '/../Views/' . $view . '.php';

        if (!$useLayout) {
            require $content;
            return;
        }

        require __DIR__ . '/../Views/layouts/main.php';
    }

    protected function redirect(string $path): void
    {
        header('Location: ' . BASE_URL . $path);
        exit;
    }

    protected function flash(string $type, string $text): void
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        $_SESSION['flash_message'] = ['type' => $type, 'text' => $text];
    }
}
```

b. MahasiswaController yang mewarisi BaseController

MahasiswaController memakai extends BaseController sehingga dapat langsung memanggil `$this->view()`, `$this->redirect()`, dan `$this->flash()` tanpa menuliskannya kembali. Seluruh akses data dilakukan melalui `$this->repo`.

```
class MahasiswaController extends BaseController
{
    private MahasiswaRepository $repo;

    public function __construct(MahasiswaRepository $repo)
    {
        $this->repo = $repo;
    }

    public function index(): void
    {
        $keyword = trim($_GET['q'] ?? '');

        $this->view(
            'mahasiswa/index',
            [
                'daftarMahasiswa' => $this->repo->all($keyword),
                'keyword' => $keyword,
            ],
            'mahasiswa'
        );
    }

    public function delete($id): void
    {
        try {
            $this->repo->deleteByNim((string) $id);
            $this->flash('success', 'Data mahasiswa berhasil dihapus.');
```

c. MahasiswaRepository (Repository Pattern)

Seluruh query SQL untuk data mahasiswa berada di MahasiswaRepository. Controller hanya memanggil method seperti `all()`, `findByNim()`, `create()`, `updateByNim()`, dan `deleteByNim()`.

```
public function all(string $keyword = ''): array
{
    if ($keyword !== '') {
        $stmt = $this->db()->prepare(
            "SELECT m.nim, m.nama, m.angkatan, m.status, p.nama AS prodi_nama
```

```

        FROM mahasiswa m JOIN prodi p ON m.prodi_id = p.id
        WHERE m.nim LIKE :keyword OR m.nama LIKE :keyword
        ORDER BY m.nim ASC"
    );
    $stmt->execute(['keyword' => '%' . $keyword . '%']);
} else {
    // query yang sama tanpa filter pencarian
}

// setiap baris hasil query diubah menjadi objek Mahasiswa
}

public function deleteByNim(string $nim): bool
{
    $stmt = $this->db()->prepare("DELETE FROM mahasiswa WHERE nim = :nim");

    return $stmt->execute(['nim' => $nim]);
}

```

d. BaseModel

BaseModel (app/Core/BaseModel.php) menyediakan method db() yang mengembalikan PDO dari class Database (Acara 9). Model seperti Prodi, Matakuliah, dan Dosen cukup mewarisinya sehingga tidak perlu membuat koneksi sendiri.

```

abstract class BaseModel
{
    private static ?Database $database = null;

    protected static function db(): PDO
    {
        if (self::$database === null) {
            self::$database = new Database();
        }

        return self::$database->getConnection();
    }
}

class Prodi extends BaseModel
{
    public static function all(): array
    {
        return self::db()->query("SELECT * FROM prodi ORDER BY kode ASC")
            ->fetchAll(PDO::FETCH_ASSOC);
    }
}

```

3. Perbandingan Acara 9 dan Acara 10

Tabel 1. Perbandingan struktur kode sebelum dan sesudah penerapan inheritance

Aspek	Acara 9	Acara 10
Method redirect() dan flash()	Ditulis ulang di tiap Controller	Ditulis sekali di BaseController dan diwarisi
Memanggil tampilan	<code>\$this->render(__DIR__ . '../Views/mahasiswa/index.php', ...)</code>	<code>\$this->view('mahasiswa/index', ...)</code>
Controller yang mewarisi class induk	Semua kecuali AuthController	Semua Controller (Mahasiswa, Prodi, Matakuliah, Dosen, Home, Auth)
Koneksi database pada Model	Class Model membuat PDO sendiri	BaseModel memakai class Database

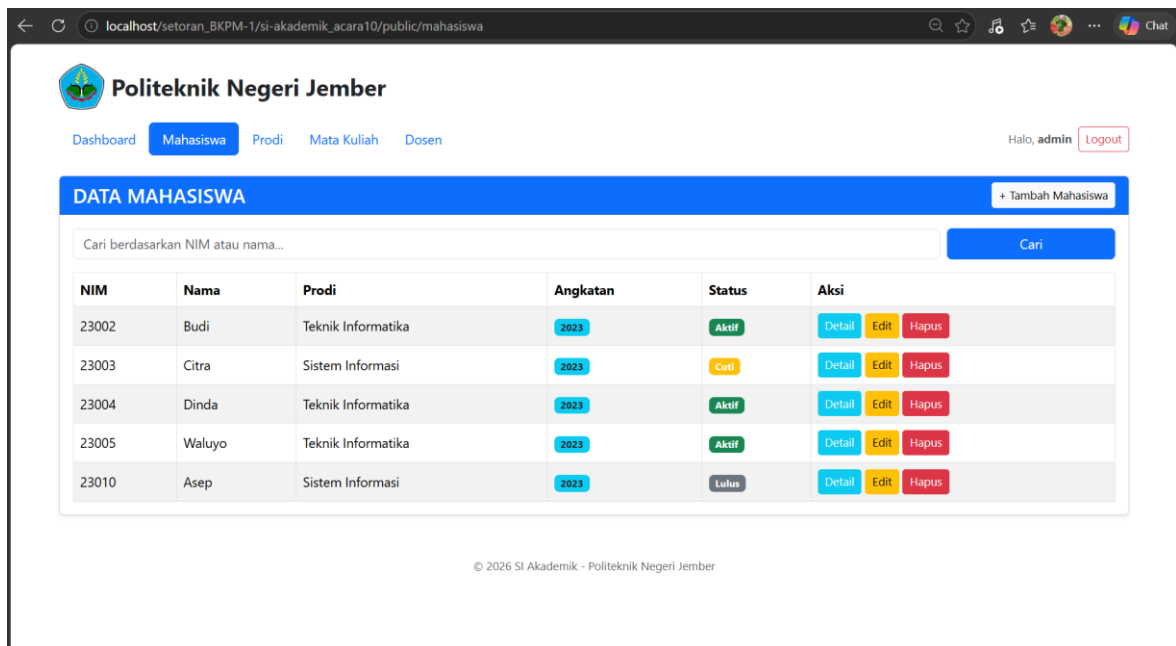
4. Pemetaan dengan Tugas Praktikum

Tabel 2. Pemetaan tugas pada modul dengan implementasi

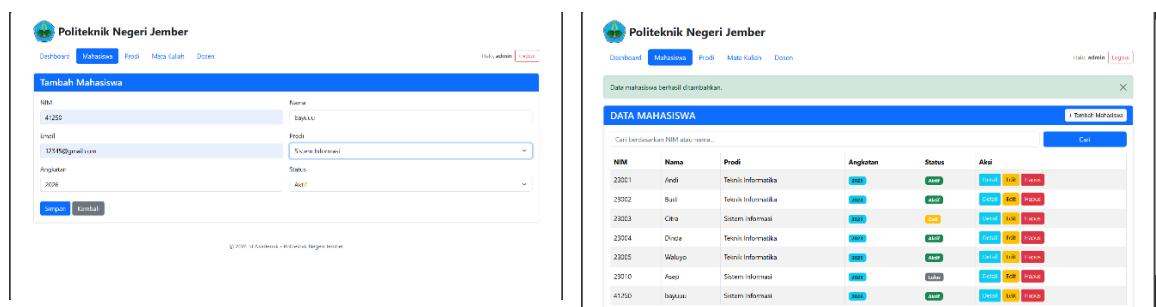
No	Tugas pada modul	Implementasi	File
1	Membuat BaseController dengan method view() dan redirect()	BaseController dengan view(), redirect(), dan flash()	app/Core/BaseController.php
2	MahasiswaController mewarisi BaseController dan memakai method yang diwariskan	extends BaseController; memakai \$this->view() dan \$this->redirect()	app/Controllers/MahasiswaController.php
3	MahasiswaRepository menangani seluruh operasi database	Method all, findByNim, create, updateByNim, deleteByNim	app/Repositories/MahasiswaRepository.php
4	Controller mengakses data hanya lewat Repository, tanpa SQL	Controller hanya memanggil \$this->repo; SQL hanya ada di Repository	app/Controllers/MahasiswaController.php
5	Menguji tampil, tambah, ubah, dan hapus data	Diuji pada browser (lihat bagian 5)	-

5. Pengujian Aplikasi

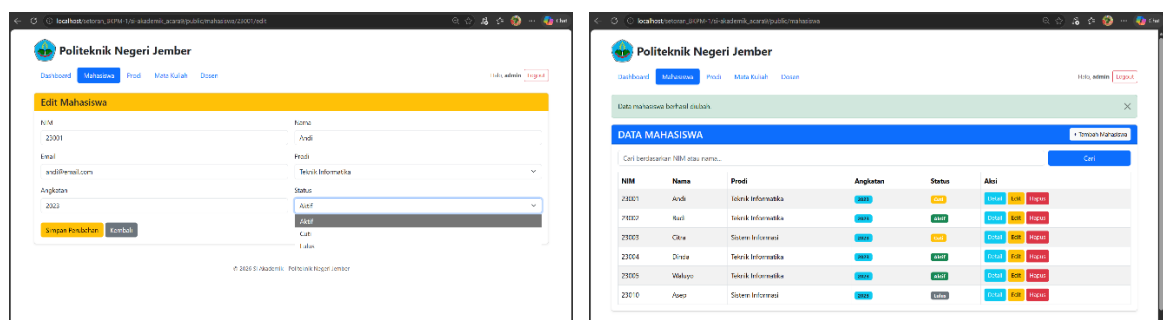
Acara 10 merupakan refactoring struktur kode, sehingga tampilan aplikasi tidak berubah dibandingkan Acara 9. Pengujian dilakukan untuk memastikan seluruh fitur tetap berjalan setelah struktur kode dirapikan.



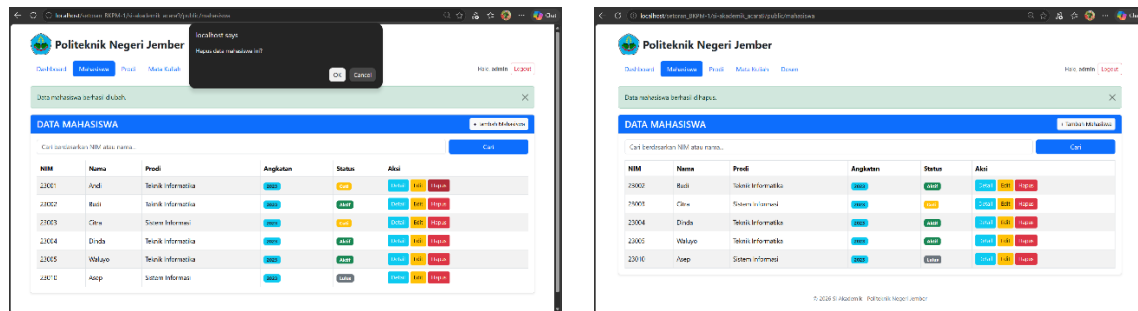
Gambar 1. Tampil data mahasiswa



Gambar 2. Tambah data mahasiswa



Gambar 3. Ubah data mahasiswa



Gambar 4. Hapus data mahasiswa

6. Pembahasan

Penerapan BaseController menghilangkan duplikasi method `redirect()` dan `flash()` yang sebelumnya ada di beberapa Controller. Jika cara mengarahkan pengguna atau menyimpan pesan perlu diubah, perubahan cukup dilakukan di satu tempat dan langsung berlaku untuk semua Controller yang mewarisinya.

Repository Pattern memisahkan logika akses database dari logika aplikasi. MahasiswaController tidak lagi berisi query SQL dan hanya memanggil method repository, sehingga kode Controller menjadi lebih sederhana dan lebih mudah dipelihara. Apabila struktur tabel berubah, perubahan cukup dilakukan pada MahasiswaRepository.

BaseModel memakai class Database yang dibuat pada Acara 9, sehingga seluruh aplikasi memakai satu jalur pembuatan koneksi dan tidak ada lagi kode koneksi yang ditulis ulang di class Model.

KESIMPULAN

1. BaseController berhasil mengurangi duplikasi kode karena method `view()`, `redirect()`, dan `flash()` ditulis sekali dan diwarisi oleh semua Controller.
2. MahasiswaRepository memusatkan seluruh query SQL sehingga MahasiswaController hanya memanggil method repository tanpa menuliskan SQL.
3. BaseModel menyatukan akses PDO pada Model dengan memakai class Database.
4. Fitur tampil, tambah, ubah, dan hapus data tetap berjalan dengan baik setelah penerapan inheritance dan repository pattern.

DAFTAR PUSTAKA

Pratama, R. A., & Hasanah, Q. (2026). Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Jurusan Teknologi Informasi, Politeknik Negeri Jember.

